Lab 7: Serial Pattern Guard Integration

Digital Design (CS F215)

26 September 2025

1 Question 1: Gated D Flip-Flop with Scrub

Goal. Build `gated_dff`, a positive-edge triggered flip-flop that supports both a synchronous load and a synchronous scrub (clear) in addition to an asynchronous active-low reset rst_n.

Theory task.

- Extend the characteristic table to include the new control signals. A typical row for controls is shown below; fill in all eight possibilities for $(rst\_n, scrub, load)$ in your notebook.

rst_n	$scrub$	$load$	d	q^+
1	0	1	d	d
1	1	X	X	d

2 Question 2: Dual-Pattern Sliding-Window Logic

Objective

You are asked to design a purely combinational logic block, called `pattern_next_logic`, that continuously observes a running serial bitstream. At any given clock cycle, the circuit receives:

- The most recent three stored bits (`curr_state[2:0]`).
- The newest incoming serial bit (`serial_in`).

Using this information, the block should determine whether the 4-bit "window" of bits reveals a target pattern.

Patterns to Detect

The goal is to recognise two specific sequences:

1. Pattern `1101`
2. Pattern `1011`

In addition, you must also anticipate "almost matches." That is, if the current window is:

- `1100` (which would become `1101` if the next input were a '1'),
- or `1010` (which would become `1011` if the next input were a '1'),

then your circuit should flag this situation as a *pre-match*.

Behavioural Requirements

- The block must compute a `next_state` that represents what the three most recent bits will be in the next cycle (after the current `serial_in` has been shifted in).
- If the current 4-bit window equals one of the two target patterns, you must set the signal `match_now` to 1.
- If the current 4-bit window equals one of the "almost match" cases, you must set the signal `pre_match` to 1.

Key Design Considerations

1. **Sliding-Window Concept.** Think of the design as always maintaining the last four bits seen. Each new input bit shifts the window to the left and brings in the fresh bit on the right.
2. **Overlap Handling.** Ensure that overlapping sequences are correctly detected. For example, the bitstream `11011` contains two matches back-to-back: the first four bits form `1101`, and then the next four bits form `1011`.
3. **State Update.** The `next_state` output is essentially the three rightmost bits of the current 4-bit window. This ensures continuity so that the detector can move smoothly into the next cycle.

Illustrative Examples

Example 1 (Direct Match). Suppose the stored state is `110` and the next input bit is `1`. The resulting window is `1101`, which is a full match. The block should raise `match_now` and also compute the next state as `101`.

Example 2 (Overlap). Consider the stream `11011`. At the fourth bit, the window is `1101`, producing a match. After shifting, the next window becomes `1011`, which is also a match. Thus two consecutive matches are observed due to overlapping.

Example 3 (Pre-Match). Suppose the stored state is `110` and the next input is `0`, giving a window of `1100`. Although no match occurs now, the `pre_match` output should be raised. If the following bit were '1', then the window would become `1101`, a full match.

3 Question 3: Masked State Register Bank

Goal. Assemble `state_register3` by instantiating three copies of `gated_dff`. It stores a state while allowing individual bits to freeze through $hold\_mask[2:0]$ and providing a synchronous scrub input.

Theory task.

- Sketch timing diagrams illustrating: (a) a bit frozen by $hold\_mask[i] = 1$ while others update, (b) the effect of a scrub pulse mid-execution.

Implementation guidance.

- Generate each flip-flop's load as $load \vee hold\_mask[i]$, feeding back q_i when the mask is 1.

Example. With $hold\_mask = 101$, bits 2 and 0 retain their present values while bit 1 accepts the new next-state value. This feature is critical when the top-level controller pauses state evolution but still wants to log history.

4 Question 4: Burst-Aware Saturating Counter

Goal. Implement `saturating_counter` that counts matches up to 3 using either single increments (inc_single) or double increments (inc_double).

Implementation guidance.

- On each tick, clear to 0 if $clr = 1$.
- Else if $inc\_double = 1$ and $count < 3$, add 2 but clamp to 3.
- Else if $inc\_single = 1$ and $count < 3$, add 1.
- Drive at_max as $(count = 3)$.

Example. Starting from $count = 1$, if $inc\_single = 1$ and $inc\_double = 1$ together, the counter must leap to 3 (not 2 or wrap to 0). The next cycle with either increment still keeps $count = 3$ until cleared.

5 Question 5: Pattern Guard Integration

Goal. Combine every previous block into `pattern_guard_top` that monitors the serial input, counts detections, tracks pre-match events, and maintains a four-bit history window.

Design checklist.

- Feed $curr\_state$ into `pattern_next_logic`; route its $next\_state$, $match\_now$, and pre_match outputs.
- Freeze the state by setting $hold\_mask = \{3, 3, 3\}$ copies of $hold\_state$ inside `state_register3`; scrub the state whenever $clear\_counter$ or $scrub\_history$ asserts.
- Drive `saturating_counter` with $inc\_single = match\_now \wedge \neg hold\_state$ and $inc\_double = pre\_match \wedge \neg hold\_state \wedge \neg match\_now$; clear it with $clear\_counter$.
- Construct the four-bit history window from four `gated_dff` instances shifting on each new bit while $freeze\_history = 0$ and clearing on $scrub\_history = 1$.

Example walk-through. Assume the serial stream delivers $\ldots 1101$ while $hold\_state = 0$ and $freeze\_history = 0$:

1. `pattern_next_logic` spots window 1101, so $match\_now = 1$ and $pre\_match = 0$.
2. `state_register3` loads the new state 101 on the clock edge.
3. `saturating_counter` increments by one; if it previously held 2, it now reaches the saturation value 3 and raises at_max.
4. The history shift register appends the new bit, so the four-bit window reported to software matches the detected pattern.